

Umbrella Notes — Module 5

Digital Logic + Computer Organization & Architecture | shared core for Coal India MT (Systems), IBPS SO IT (SPL-XVI), DSSSB TGT CS (DOE)

How to read this module

This is **full teaching notes** — learn directly from it. Depth is **balanced across all three exams** (middle depth by default: enough for IBPS SO, DSSSB TGT, and general PSU/Govt CS-IT-Systems exams).

■ **Green box** = how PSU/Govt exams actually test the topic (a reference lens, not a limit).

↑ **Amber box** = extra GATE-core depth needed *only* if targeting Coal India MT (or GATE-style PSUs). Skip for IBPS/DSSSB.

■ **Purple box** = a coverage warning (something kept light, or where an exam needs more/less).

Small sub-topics are folded under umbrella headings but never dropped. Topic-wise Q&A with explanations comes as a separate set after your walkthrough.

0 What this module covers

This module bundles two closely-related mid-size topics. **Digital Logic** is the bottom layer of computing — how 0s and 1s are represented and combined by gates to build circuits. **Computer Organization & Architecture (COA)** builds on that to explain how a CPU, memory, and I/O are structured and work together. The link: **logic gates** → **circuits** → **the processor**.

- **Part A — Digital Logic:** number systems, Boolean algebra, logic gates, combinational & sequential circuits.
- **Part B — COA:** CPU organization, instruction cycle, addressing modes, memory hierarchy & cache, I/O (interrupt & DMA), pipelining.

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

For IBPS SO IT and DSSSB, the high-yield bits are **number-system conversions**, **logic gate truth tables**, **memory types (RAM/ROM)**, and **memory hierarchy order**.

For Coal India / GATE this is a heavier area — K-maps, flip-flops, addressing modes, cache math and pipelining carry real weight (flagged in amber throughout).

PART A Digital Logic

A.1 Number systems

Computers work in **binary** (base 2). You must convert fluently between bases:

System	Base	Digits
Binary	2	0, 1
Octal	8	0-7
Decimal	10	0-9
Hexadecimal	16	0-9, A-F

- **To decimal:** multiply each digit by its positional weight ($\text{base}^{\text{position}}$) and sum.
- **From decimal:** repeated division by the base; remainders (bottom-up) give the answer.
- **Binary $\leftrightarrow$ octal/hex shortcut:** group binary digits in 3s (octal) or 4s (hex).
- **1 nibble = 4 bits, 1 byte = 8 bits = 2 hex digits.**

A.2 Signed numbers & complements

Negative numbers use **2's complement** (invert all bits, add 1) — the standard because it makes subtraction into addition and has a single representation of zero. **1's complement** just inverts the bits. Range of an n-bit 2's-complement number: $-2^{(n-1)}$ to $+2^{(n-1)}-1$.

A.3 Boolean algebra & gates

Gate	Symbol logic	Output is 1 when...
AND	$A \cdot B$	both inputs are 1
OR	$A + B$	at least one input is 1
NOT	A'	input is 0 (inverter)
NAND	$(A \cdot B)'$	NOT of AND — universal gate
NOR	$(A + B)'$	NOT of OR — universal gate
XOR	$A \oplus B$	inputs differ
XNOR	$(A \oplus B)'$	inputs are equal

■ Two facts exams reward

NAND and NOR are 'universal gates' — any circuit can be built from only NANDs (or only NORs). Very common MCQ.

De Morgan's laws: $(A \cdot B)' = A' + B'$ and $(A + B)' = A' \cdot B'$. Used constantly to simplify expressions.

A.4 Combinational vs sequential circuits

	Combinational	Sequential
Output depends on	Present inputs only	Present inputs + past state (memory)
Memory element	None	Flip-flops / latches
Examples	Adder, multiplexer, decoder, encoder	Registers, counters, flip-flops

- **Combinational blocks:** **half/full adder** (binary addition), **multiplexer (MUX)** (many inputs $\rightarrow$ one output, selected by control lines), **demultiplexer**, **encoder/decoder**.
- **Sequential elements:** **flip-flops** (SR, D, JK, T) store one bit; **registers** store multiple bits; **counters** count clock pulses.

↑ **Go deeper — GATE-core (Coal India MT)**

Coal India / GATE: simplify Boolean expressions with **K-maps** (up to 4 variables, don't-cares), design combinational circuits, analyse **flip-flop excitation tables** and build counters, and do **signed binary arithmetic** (overflow detection, floating-point IEEE 754). IBPS/DSSSB need gate truth tables + number conversions, not full circuit design.

PART B Computer Organization & Architecture

B.1 The big picture — von Neumann model

Most computers follow the **von Neumann architecture**: a single memory holds both **instructions and data**, and a CPU fetches and executes them one at a time. Its parts: **CPU** (control unit + ALU + registers), **memory**, and **I/O**, connected by **buses**.

- **Control Unit (CU)** — directs operations, decodes instructions, generates control signals.
- **ALU** — performs arithmetic & logic operations.
- **Registers** — tiny, fastest storage inside the CPU (e.g. **PC** program counter, **IR** instruction register, **MAR/MDR**, **accumulator**).
- **Buses** — **address bus** (carries addresses, one-way), **data bus** (carries data, two-way), **control bus** (control signals). Address-bus width sets how much memory is addressable (2^n).

■ Von Neumann vs Harvard

Von Neumann — one shared memory + bus for instructions and data (simpler, but the 'von Neumann bottleneck').

Harvard — separate memory/buses for instructions and data (faster, used in DSP/microcontrollers). A common comparison MCQ.

B.2 Instruction cycle & addressing modes

The CPU repeats the **instruction cycle**: **Fetch** → **Decode** → **Execute** (with optional operand fetch / interrupt check). **Addressing modes** specify where an instruction's operand is:

Mode	Operand is...
Immediate	A constant given in the instruction itself
Direct	At the memory address given in the instruction
Indirect	At the address stored at the given memory location
Register	In a named register
Register indirect	At the address held in a register
Indexed / Base	Base address + an index offset

B.3 Memory hierarchy

Memory trades **speed vs cost vs size**. From fastest/smallest/costliest at the top to slowest/largest/cheapest at the bottom:

Level	Type	Note
Registers	Inside CPU	Fastest, tiny
Cache (L1/L2/L3)	SRAM	Very fast; holds recently-used data
Main memory (RAM)	DRAM	Volatile working memory
Secondary storage	SSD / HDD	Non-volatile, large, slow

- **RAM** (volatile, loses data on power-off): **SRAM** (fast, cache) vs **DRAM** (denser, main memory, needs refresh).
- **ROM** (non-volatile): PROM, EPROM, EEPROM — stores firmware/BIOS.
- **Cache** works because of **locality of reference** (programs reuse nearby data/instructions). A **cache hit** = found in cache; a **miss** = must fetch from lower level.

■ Tested pattern & focus (PSU / Govt CS-IT-Systems)

Near-certain MCQs: **RAM vs ROM** (volatile vs non-volatile), **SRAM vs DRAM**, the **memory hierarchy order** (register > cache > RAM > disk by speed), and which memory is **volatile**.

Also common: purpose of specific registers (PC holds next instruction address; IR holds current instruction) and the fetch-decode-execute cycle.

B.4 I/O & interrupts

- **Programmed I/O** — CPU polls the device (wastes CPU time).
- **Interrupt-driven I/O** — device signals the CPU when ready (CPU free meanwhile).
- **DMA (Direct Memory Access)** — a controller transfers data directly between device and memory, bypassing the CPU for the bulk transfer (efficient for large blocks).

↑ Go deeper — GATE-core (Coal India MT)

Coal India / GATE: **cache mapping** (direct / associative / set-associative) and hit-ratio + effective-access-time math, **pipelining** (stages, speedup = k-stage ideal, **hazards**: structural/data/control), instruction formats, and micro-programmed vs hardwired control. This is substantial for Coal India and out of scope for IBPS/DSSSB.

Rapid recap — highest-yield facts

Area	Nail these
Number systems	Binary/octal/decimal/hex conversions; nibble=4 bits, byte=8 bits=2 hex; 2's complement (invert+1) for negatives
Gates	AND/OR/NOT/NAND/NOR/XOR/XNOR truth tables; NAND & NOR are universal; De Morgan's laws
Circuits	Combinational(inputs only: adder/MUX/decoder) vs sequential(inputs+state: flip-flops/counters/registers)
CPU	Von Neumann (shared mem) vs Harvard (separate); CU+ALU+registers; PC/IR/MAR/MDR; fetch-decode-execute
Buses	Address(1-way, sets 2^n addressable), data(2-way), control
Memory	Hierarchy register>cache>RAM>disk; RAM volatile / ROM non-volatile; SRAM(cache) vs DRAM(main); locality; hit/miss
I/O	Programmed(poll) vs interrupt-driven vs DMA(bypass CPU for bulk)

✓ Before your walkthrough

That's the shared-core Digital Logic + COA picture at balanced depth. Flag any page/section that felt shaky and I'll go deeper; then the **topic-wise Q&A set** (real PSU/Govt questions + explanations, incl. number-conversion and truth-table problems) follows for live practice.

Coal India target? This is a heavy area for you — work all amber ↑ **GATE-core** boxes as a full second pass: K-maps, flip-flop design, addressing-mode problems, cache mapping/hit-ratio math, and pipelining hazards.